Lab 3

Threads and a shared queue

SENTINEL Lab 3: libipc. Many worker threads in one process, on one processor.

Where Lab 3 fits

In Lab 2 the centre handled each report itself. One thread cannot keep up with SAT, at 2000 reports each second. Lab 3 adds worker threads.

Lab 1 · Oct. 9

libsyscall

The interface to the operating system: the feed, the mission log and precise timestamps.

R6

Read a feed and lose none of it. Put a real date on each report. Record each report, in order.

Lab 2 · Oct. 30

libproc

One process for each sensor, and the channels that bring their reports to the centre.

R1 R2

Continue when one sensor stops. Let an urgent report overtake routine traffic.

Lab 3 · Nov. 20 · this lab

libipc

Worker threads that share one processor and keep the track picture.

R3

Take thousands of reports each second. Measure what shared data with no lock costs.

Lab 4 · Dec. 4

libsync

Protection for each thing that the workers share.

R4 R5

Keep the picture exact while many workers change it. Never freeze.

Lab 3 at a glance

Nov. 18

Prelab due by midnight.
Necessary, no marks.

Nov. 20

Lab session. The Lab 2 oral exam is in this session.

Dec. 2

Deliverable due by midnight.
80 marks, 5% of the grade.

Dec. 4

Oral exam in the Lab 4 session.
20 marks, 3% of the grade.

The 80 marks of the deliverable

20	24	12	12	4	8
Acceptance test	Course tests	Your test	Code rules and errors	valgrind, threads joined	Two files agree

What you write in Lab 3

`src/libipc/src/queue.c`	queue_init, queue_destroy, queue_push, queue_pop, queue_count
`src/libipc/src/pool.c`	pool_create_deadline, pool_stop, pool_destroy
`src/libipc/tests/test_queue.c`	Three test cases, with assert(), in one thread

Read these first. Do not change them

`ipc.h`

The steps, the errors and the log lines of each function.

`given/pool_watch.c`

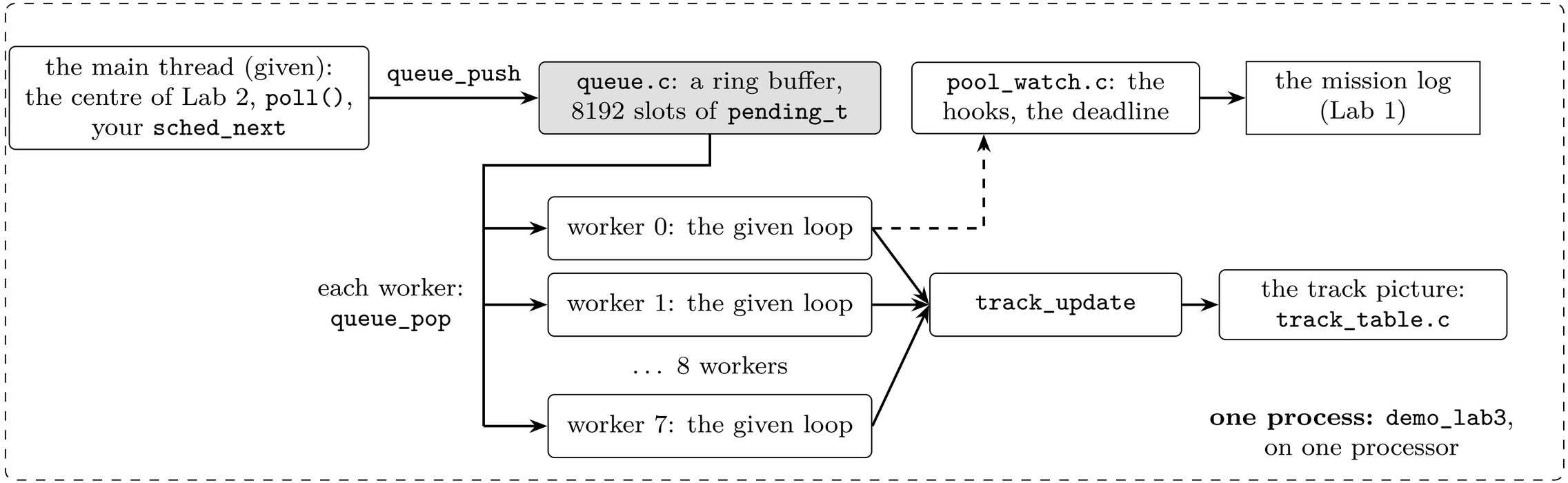
The worker loop that each of your threads runs.

`given/track_table.c`

The track picture. Each worker calls track_update.

Do not add a lock, an atomic variable or a semaphore. Lab 3 measures the code with no protection.

How Lab 3 works



Grey: your code. White: given code. Square corners: a file. The queue and the track picture are shared, with no lock.

The ideas of Lab 3

Threads share one memory

The workers are threads of one process, so each worker sees at once each change that another worker makes to the queue.

`Brief 3.1`

Give each worker its own block

Give each worker the address of workers[i], not of the loop variable i: the loop changes i before the worker can read it.

`Brief 3.2`

The queue is a ring buffer

The queue is a fixed array: after the last slot it goes back to slot 0, and count tells if the queue is empty or full.

`Brief 3.3`

A full queue refuses a report

When the queue is full, queue_push refuses the report with ENOSPC and counts it, so each refused report has a record.

`Brief 3.3`

The stop empties the queue first

pool_stop tells the workers to stop; each worker empties the queue first, and pthread_join() waits until each worker returns.

`Brief 3.4`

One processor, many workers

On one processor the 8 workers take turns, so together they never get more than the time of one processor.

`Brief 3.6`

Three steps, not one

`t->total_reports++ is LOAD, ADD 1, STORE`

Two workers can both LOAD 5 and both STORE 6. One update is lost, and no error occurs. That is a race. Your queue_pop has the same three steps on count.

8 workers, 10 runs of 10 s each	Agrees	DISAGREES	DID NOT STOP
`--cpus 1, one processor`	**10**	**0**	**0**
`--cpus 2`	**0**	**2**	**8**
`--cpus all, 8 processors`	**0**	**3**	**7**

A run that gives the right total proves nothing.

Run it

Run each command at the top of your repository.

`make`	Builds src/libipc. Each stub fails until you write it.
`make lab3`	Runs the demo with 8 workers for 3 s. WORKERS=1 and CPUS=all change the run.
`make test LAB=3`	Runs your test, with the tests of Labs 1 and 2.
`make accept3`	Runs the 14 checks of the acceptance test.
`make submit LAB=3`	Writes the zip of Lab 3.

Done when, then hand in

- [ ] make prints no warning, and make test LAB=3 passes.
- [ ] make accept3 gives 14 of 14.
- [ ] demo_1worker.txt says BALANCED, and the shared data agrees.
- [ ] demo_8workers.txt says DISAGREES or DID NOT STOP.
- [ ] valgrind reports no error and no leak.
- [ ] Your code has no lock, no atomic variable and no semaphore.
- [ ] make accept2 still gives 26 of 26.

Then hand in

```
make submit LAB=3
sentinel-lab3-<name>.zip
```

Put the two files of the record runs in docs/lab3/. The 8-worker run needs 2 processors or more: nproc gives the number.

Lab 3 Brief 6 gives the commands.

Upload the zip to Brightspace by Dec. 2, midnight.

Questions?

If you have no questions, start work on Lab 3.

Read these two files first:

`docs/lab3/lab3_brief.pdf`

`src/libipc/include/sentinel/ipc.h`